

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

3

PHASE THREE

Pydantic: Data Validation

Turn the quick taste of Pydantic into bulletproof validation: field constraints, request and response models, the Create/Update/Response pattern, custom validators, nested models, and clean error handling.

Contents

3.1 Why Data Validation Matters	4
3.2 Pydantic v2 Basics	5
Defining a model	5
Common field types	6
Adding constraints with Field()	6
Optional fields	7
3.3 Request Models	7
3.4 Response Models	8
3.5 Model Inheritance and Reuse	9
3.6 Custom Validators	10
@field_validator — validate one field	11
@model_validator — validate across fields	11
3.7 Nested Models	12
3.8 Error Handling	13
Raising errors with HTTPException	13
What a 422 validation error looks like	14
Custom global exception handlers	14
Hands-On Exercise 3: Rebuild the Book Library with full Pydantic	15
Checkpoint: At This Point You Can Now...	18
Phase 3 Summary	19

Your Journey

✓	Foundations: What Is an API?	DONE
✓	Setup & Your First API	DONE
✓	Path Params, Query Params & Body	DONE
3	Pydantic: Data Validation	YOU ARE HERE
4	Database Integration (SQLAlchemy)	
5	Authentication & Authorization	
6	Advanced FastAPI Features	
7	Testing Your FastAPI App	
8	Deployment with Docker	
★	Capstone — TaskFlow API	

In Phase 2 you used a Pydantic model to receive a request body and it "just worked." This phase is about understanding *why* it worked and how to wield it fully. Pydantic is the beating heart of FastAPI: it's the library that turns messy, untrusted JSON from the outside world into clean, typed, validated Python objects you can trust. Master Pydantic and you've mastered most of what makes FastAPI feel magical. Everything here uses **Pydantic v2**, the current major version.

3.1 Why Data Validation Matters

Your API is a door to the outside world, and the outside world sends garbage — sometimes by accident, sometimes maliciously. A client might send a string where you expect a number, omit a required field, send a negative age, or include a 10,000-character "name" designed to break your database. **Validation is the bouncer at the door:** it checks every piece of incoming data against rules *before* that data reaches your logic or your database.

Without validation, the consequences range from annoying to catastrophic:

- A `"year": "soon"` string crashes your code with a `TypeError` deep inside business logic, returning an ugly `500`.
- An unvalidated field flows straight into a database query, opening the door to **injection attacks**.
- Inconsistent data (some books with a `year`, some without) quietly corrupts your dataset and breaks features months later.

Validation is also a **contract**. It states, precisely and in code, "here is exactly what I accept and what I return." FastAPI chose Pydantic for this because Pydantic expresses that contract using plain Python type hints — the same hints that also generate your docs and your editor autocomplete. One definition, three payoffs: validation, documentation, and tooling.

CONCEPT CHECK

Concept Check: A client sends `{"title": "Dune", "year": "1965"}` (year as a *string*, not a number). With `year: int` in your model, what does Pydantic do? *Answer: It tries a safe coercion — the string "1965" becomes the integer 1965 because it's a clean numeric string. But "long ago" would fail with a 422. Pydantic is helpful where it safely can be, strict where it must be.*

3.2 Pydantic v2 Basics

Defining a model

A Pydantic model is a class that inherits from `BaseModel`. Each annotated attribute becomes a validated field:

PYTHON

```
from pydantic import BaseModel

class Book(BaseModel):
    title: str          # required string
    author: str          # required string
    year: int           # required integer
    in_stock: bool = True # optional, defaults to True
```

Create one by passing data; Pydantic validates on construction:

PYTHON

```
# Valid – note "1965" (string) is safely coerced to int 1965.
book = Book(title="Dune", author="Herbert", year="1965")
print(book.year)          # 1965 (an int)
print(book.model_dump())  # {'title': 'Dune', 'author': 'Herbert', 'year': 1965,
                          # 'in_stock': True}
```

Common field types

Pydantic understands a rich set of types out of the box:

Type	Accepts / meaning
<code>str</code> , <code>int</code> , <code>float</code> , <code>bool</code>	The basics, with safe coercion
<code>datetime</code> , <code>date</code> , <code>time</code>	Parsed from ISO strings like <code>"2024-01-15"</code>
<code>UUID</code>	Validated UUID strings
<code>list[str]</code> , <code>dict[str, int]</code>	Typed collections
<code>EmailStr</code>	A validated email address (needs <code>pip install pydantic[email]</code>)
<code>X None</code>	Optional — the value may be missing/null

Adding constraints with `Field()`

`Field()` lets you attach rules and metadata to a field — defaults, descriptions, examples, and min/max constraints that Pydantic enforces and FastAPI shows in the docs:

PYTHON

```
from pydantic import BaseModel, Field

class Book(BaseModel):
    # min_length / max_length constrain string size.
    title: str = Field(min_length=1, max_length=200,
                      description="The book's title")
    # ge = "greater than or equal", le = "less than or equal".
    year: int = Field(ge=1000, le=2025, description="Year of publication")
    # A default plus a description.
    rating: float = Field(default=0.0, ge=0, le=5)
```

Now `title=""` (empty) or `year=3000` is rejected automatically with a clear `422`, and `/docs` displays the constraints and descriptions.

TIP

Tip — the common numeric constraints: `gt` (greater than), `ge` (greater-or-equal), `lt` (less than), `le` (less-or-equal). For strings: `min_length`, `max_length`, and `pattern` (a regex). These cover the vast majority of real validation needs without writing any custom code.

Optional fields

In Python 3.11, the clean way to mark a field optional is `Type | None` with a default:

PYTHON

```
class Book(BaseModel):
    title: str
    subtitle: str | None = None  # may be omitted; defaults to None
```

COMMON MISTAKES

Common Mistakes: Writing `subtitle: str | None` *without* a default. That still makes the field **required** (you'd have to explicitly pass `None`). To make it truly optional, add `= None`. The type says "can be null"; the default says "can be missing" — you usually want both.

3.3 Request Models

A **request model** describes the JSON shape a client may send in a POST/PUT/PATCH body. You met this in Phase 2: define the model, type the body argument with it, and FastAPI parses → validates → injects a clean object.

PYTHON

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class BookCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    author: str
    year: int = Field(ge=1000, le=2025)

@app.post("/books")
def create_book(book: BookCreate):
    # `book` is a fully-validated BookCreate; if we got here, the data is good.
    return {"created": book.title, "year": book.year}
```

What FastAPI does under the hood, in order: reads the raw request body → parses it as JSON → validates it against `BookCreate` → if valid, constructs the object and calls your function; if invalid, short-circuits with a `422` and never runs your code. Your function body can therefore *assume the data is clean*.

3.4 Response Models

You can also declare what an endpoint **returns** using the `response_model` parameter. This does three valuable things: documents the response shape in `/docs`, validates your own output, and — crucially — **filters out fields you don't want to expose**.

PYTHON

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str      # sensitive!

class UserOut(BaseModel):
    username: str      # note: no password field

@app.post("/users", response_model=UserOut)
def create_user(user: UserIn):
    # Even though we return an object containing the password, FastAPI
    # filters the response through UserOut, so the password is NEVER sent back.
    return user
```

Send a user with a password, and the response contains only `username`. The password is stripped automatically because `UserOut` has no such field. This is the standard, safe way to ensure secrets never leak in responses.

There are also fine-grained controls:

- `response_model_exclude={"field"}` — drop specific fields from the response.
- `response_model_include={"field"}` — include *only* specific fields.
- `response_model_exclude_none=True` — omit fields whose value is `None`.

TIP

Tip: Always use a dedicated response model for anything touching users or auth. "We'll remember to strip the password manually" is exactly the kind of promise that gets forgotten in a late-night hotfix. Let the `response_model` enforce it structurally so it can never leak.

CONCEPT CHECK

Concept Check: Why validate your own output with a response model — isn't your code trustworthy? *Answer: It catches mistakes. If a refactor accidentally returns the wrong shape or a sensitive field, the response model flags or filters it. It's a safety net for your own bugs, not just the client's.*

3.5 Model Inheritance and Reuse

Real entities need several *views*: the fields a client sends to **create** one, the (often partial) fields to **update** one, and the fields you **return** (which include server-generated values like `id` and `created_at`). Re-typing every field in every model is repetitive and error-prone. The professional pattern is a small inheritance hierarchy built on a shared base.

PYTHON

```
from pydantic import BaseModel, Field
from datetime import datetime

# Shared fields live in the base – written once.
class UserBase(BaseModel):
    username: str = Field(min_length=3, max_length=50)
    email: str

# Create: base fields + the things only sent at creation.
class UserCreate(UserBase):
    password: str = Field(min_length=8)

# Update: every field optional, so a client can send just what changed.
class UserUpdate(BaseModel):
    username: str | None = None
    email: str | None = None

# Response: base fields + server-generated ones; never includes password.
class UserResponse(UserBase):
    id: int
    created_at: datetime

# Lets this model be built directly from an ORM object (Phase 4).
model_config = {"from_attributes": True}
```

This **Base → Create / Update / Response** pattern is one you'll use for every entity in the capstone. It keeps shared rules in one place, makes the differences between views explicit, and ensures the password lives only on **UserCreate**, never on a response.

TIP

Tip — `from_attributes=True` : This setting (Pydantic v2's name for the old `orm_mode`) lets a Pydantic model read its values from the *attributes* of another object — like a SQLAlchemy database row — instead of a dict. You'll rely on it constantly once you add a database in Phase 4. Set it on your response models now and it'll be ready.

3.6 Custom Validators

Built-in constraints cover length and range, but sometimes you need custom logic — "this field must be a valid slug," or "these two fields must agree." Pydantic v2 gives you two decorators.

@field_validator — validate one field

PYTHON

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    username: str

    @field_validator("username")
    @classmethod
    def no_spaces(cls, value: str) -> str:
        # Runs after the basic type check. Raise ValueError to reject.
        if " " in value:
            raise ValueError("username must not contain spaces")
        return value.lower() # you can also transform the value
```

A `@field_validator` runs for the named field. Raise a `ValueError` to reject (FastAPI turns it into a clean `422`), or return a (possibly transformed) value to accept.

@model_validator — validate across fields

When a rule involves *more than one* field, use `@model_validator(mode="after")`, which runs once the whole object is built:

PYTHON

```
from pydantic import BaseModel, model_validator

class SignUp(BaseModel):
    password: str
    confirm_password: str

    @model_validator(mode="after")
    def passwords_match(self):
        # `self` is the fully-built model; compare fields freely.
        if self.password != self.confirm_password:
            raise ValueError("passwords do not match")
        return self
```

Another classic cross-field rule — a date range where the end must follow the start:

PYTHON

```
from datetime import date
from pydantic import BaseModel, model_validator

class Event(BaseModel):
    start_date: date
    end_date: date

    @model_validator(mode="after")
    def end_after_start(self):
        if self.end_date < self.start_date:
            raise ValueError("end_date must be on or after start_date")
        return self
```

COMMON MISTAKES

Common Mistakes: Forgetting the `@classmethod` line under `@field_validator`, or forgetting to **return the value** at the end. A field validator that doesn't return ends up setting the field to `None`. Always return the validated value (or `self` for model validators).

CONCEPT CHECK

Concept Check: Password-confirmation matching — field or model validator? *Answer: A `@model_validator`, because it compares two fields. A `@field_validator` only sees one field at a time.*

3.7 Nested Models

Models can contain other models, which lets you validate rich, structured JSON. A book can carry a nested `Author`; an order can hold a list of `OrderItem`s.

PYTHON

```
from pydantic import BaseModel

class Author(BaseModel):
    name: str
    birth_year: int

class Book(BaseModel):
    title: str
    year: int
    author: Author      # a nested model

class Order(BaseModel):
    order_id: int
    items: list["OrderItem"]  # a list of nested models

class OrderItem(BaseModel):
    product: str
    quantity: int
```

The matching JSON nests naturally, and Pydantic validates every level:

JSON

```
{
  "title": "Dune",
  "year": 1965,
  "author": {"name": "Frank Herbert", "birth_year": 1920}
}
```

If `author.birth_year` were `"old"`, Pydantic would reject it with a `422` whose error location points precisely at `body → author → birth_year`. Nested validation is fully recursive and free.

3.8 Error Handling

Raising errors with `HTTPException`

When *your* logic detects a problem (a missing resource, a forbidden action), raise `HTTPException` with the right status code — you used this in Phases 1-2:

PYTHON

```
from fastapi import HTTPException, status

raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="Book not found",
)
```

What a 422 validation error looks like

When Pydantic validation fails, FastAPI returns a `422` with a structured body listing every problem — which field, what went wrong, and where:

JSON

```
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": ["body", "year"],
      "msg": "Input should be greater than or equal to 1000",
      "input": 500
    }
  ]
}
```

The `loc` (location) array is your friend: `["body", "year"]` means "in the request body, the `year` field." Read it to know exactly what the client got wrong.

Custom global exception handlers

You can customise how specific errors are presented across the whole app with

`@app.exception_handler`:

PYTHON

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError

app = FastAPI()

# Override the default 422 body with your own shape.
@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=422,
        content={"error": "Validation failed", "details": exc.errors()},
    )
```

TIP

Tip: Global handlers are great for consistency — every validation error, 404, or unexpected 500 can return the same envelope shape your frontend expects. We'll set up a clean set of these in the capstone project.

HANDS-ON

Hands-On Exercise 3: Rebuild the Book Library with full Pydantic

Re-do the Book Library models with the full schema hierarchy, real constraints, a nested `Author`, and a custom cross-field validator.

Requirements:

- `BookBase`, `BookCreate`, `BookUpdate`, `BookResponse` models.
- Validate: `title` 1–200 chars; `year` between 1000 and 2025.
- A nested `Author` model with `name` and `birth_year`.
- Custom validator: the author's `birth_year` must be **before** the book's `year`.
- Demonstrate what the `422` looks like when validation fails.

Here is a complete, working solution:

```

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field, model_validator
from datetime import datetime

app = FastAPI(title="Book Library API")

# ---- Nested model ----
class Author(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    birth_year: int = Field(ge=0, le=2025)

# ---- Shared fields ----
class BookBase(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    year: int = Field(ge=1000, le=2025)
    genre: str
    author: Author          # nested

    @model_validator(mode="after")
    def author_born_before_book(self):
        # Cross-field rule spanning the nested model and the book year.
        if self.author.birth_year >= self.year:
            raise ValueError("author.birth_year must be before the book's year")
        return self

# ---- Create: just the base fields ----
class BookCreate(BookBase):
    pass

# ---- Update: everything optional (partial update) ----
class BookUpdate(BaseModel):
    title: str | None = Field(default=None, min_length=1, max_length=200)
    year: int | None = Field(default=None, ge=1000, le=2025)
    genre: str | None = None
    author: Author | None = None

# ---- Response: base + server fields ----
class BookResponse(BookBase):
    id: int
    created_at: datetime
    model_config = {"from_attributes": True}

# ---- In-memory store ----
books: dict[int, BookResponse] = {}
next_id = 1

@app.post("/books", response_model=BookResponse,
          status_code=status.HTTP_201_CREATED)
def create_book(payload: BookCreate):

```

```

global next_id
book = BookResponse(id=next_id, created_at=datetime.now(),
                    **payload.model_dump())

books[next_id] = book
next_id += 1
return book

@app.get("/books/{book_id}", response_model=BookResponse)
def get_book(book_id: int):
    if book_id not in books:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                            detail=f"Book {book_id} not found")

    return books[book_id]

```

Try a valid create (`POST /books`):

JSON

```

{
  "title": "Dune",
  "year": 1965,
  "genre": "scifi",
  "author": {"name": "Frank Herbert", "birth_year": 1920}
}

```

→ `201 Created` with an `id` and `created_at` added.

Now trigger validation failures and read the `422` :

TEXT

```

year = 500          → 422, loc ["body","year"], "greater than or equal to 1000"
title = ""          → 422, loc ["body","title"], "at least 1 character"
author.birth_year=1970, year=1965
                    → 422, "author.birth_year must be before the book's year"

```

What just happened? You built a complete, professional validation layer with zero manual `if`-checks scattered through your endpoint code. The schema hierarchy (`Base` → `Create` / `Update` / `Response`) keeps shared rules in one place and separates "what clients send" from "what we return" (note `BookResponse` adds `id` and `created_at`, which clients can't set). The nested `Author` is validated recursively, and the `@model_validator` enforces a business rule that spans two fields. Every failure produces a precise, machine-readable `422` — your endpoints can now trust their inputs completely. This is the difference between hobby code and production code.

COMMON MISTAKES

Common Mistakes: Putting server-controlled fields like `id` or `created_at` on the `create` model. If `BookCreate` had an `id`, a client could choose its own id — a real bug. Keep server-assigned fields on the **response** model only, never on create.

TIP

Tip: The `BookUpdate` model with all-optional fields is built for `PATCH` (partial update). When you apply it, use `payload.model_dump(exclude_unset=True)` to get *only* the fields the client actually sent, so you don't overwrite existing values with `None`.

Checkpoint: At This Point You Can Now...

- ☐ Explain **why validation matters** (crashes, injection, data integrity, the client/server contract).
- ☐ Define Pydantic v2 models with typed fields and `Field()` **constraints** (`min_length`, `ge`, `le`, ...).
- ☐ Mark fields **optional** correctly (`Type | None = None`).
- ☐ Use models as **request bodies** and as **response models**, including hiding sensitive fields.
- ☐ Apply the `Base → Create / Update / Response` inheritance pattern.
- ☐ Write `@field_validator` and `@model_validator` for single-field and cross-field rules.
- ☐ Build and validate **nested models**.
- ☐ Read a `422` error body (especially `loc`) and add a **global exception handler**.

Phase 3 Summary

PHASE 3 • KEY TAKEAWAYS

In this phase you made your data validation iron-clad with Pydantic v2.

- Validation is the **bouncer at the door** — it rejects bad data before it reaches your logic or database, and encodes the client/server **contract** in plain type hints.
- Models subclass `BaseModel`; `Field()` adds constraints (`min_length`, `max_length`, `ge`, `le`, `pattern`) and docs metadata; optional fields are `Type | None = None`.
- **Request models** validate incoming bodies; **response models** document output and **filter out sensitive fields** like passwords.
- The `Base → Create / Update / Response` hierarchy keeps shared rules in one place and separates what clients send from what you return.
- `@field_validator` handles one field; `@model_validator(mode="after")` handles cross-field rules (passwords match, end-after-start).
- **Nested models** validate structured JSON recursively; failures yield a precise `422` with a `loc` pointing at the exact field; global handlers customise error shapes.
- **You rebuilt the Book Library models** with constraints, a nested `Author`, and a business-rule validator — production-grade validation with no manual checks.

Your data layer is now trustworthy. Next phase, you'll give your API a memory that survives restarts: a real database with SQLAlchemy.

End of Phase 3. Reply "**continue**" for Phase 4 — Database Integration with SQLAlchemy — where the in-memory list becomes a real, persistent database.